

MySQL with Sequelize ORM

Here's a **complete project setup using MySQL + Sequelize ORM** for a simple app with:

- **User model**
 - **Blog model**
 - CRUD functionality
 - Clean folder structure
 - Sequelize ORM
-

Folder Structure

bash

CopyEdit

myapp/

```
|
|├── config/
| |└── db.js      # Sequelize DB connection setup
|
|├── models/
| |├── index.js   # Initializes Sequelize and all models
| |├── user.model.js  # User model
| |└── blog.model.js  # Blog model
|
|├── routes/
| |├── user.routes.js
| |└── blog.routes.js
|
|├── controllers/
| |├── user.controller.js
| |└── blog.controller.js
```

```
|
|└─ migrations/      # (optional) Sequelize CLI migrations
|└─ app.js           # Express app entry
|└─ package.json
└─ .env
```

1. Initialize Project

bash

CopyEdit

```
mkdir myapp && cd myapp
```

```
npm init -y
```

```
npm install express sequelize mysql2 dotenv
```

2. .env

env

CopyEdit

```
DB_NAME=node_app
```

```
DB_USER=root
```

```
DB_PASS=your_mysql_password
```

```
DB_HOST=localhost
```

3. config/db.js

js

CopyEdit

```
const { Sequelize } = require('sequelize');
```

```
require('dotenv').config();
```

```
const sequelize = new Sequelize(
```

```
  process.env.DB_NAME,
```

```
  process.env.DB_USER,
```

```
  process.env.DB_PASS,
```

```
{
  host: process.env.DB_HOST,
  dialect: 'mysql',
  logging: false,
}
);
```

```
module.exports = sequelize;
```

4. models/user.model.js

js

CopyEdit

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/db');
```

```
const User = sequelize.define('User', {
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false,
  }
});
```

```
module.exports = User;
```

5. models/blog.model.js

js

CopyEdit

```
const { DataTypes } = require('sequelize');  
const sequelize = require('../config/db');  
const User = require('../user.model');
```

```
const Blog = sequelize.define('Blog', {  
  title: {  
    type: DataTypes.STRING,  
    allowNull: false,  
  },  
  content: {  
    type: DataTypes.TEXT,  
  }  
});
```

```
Blog.belongsTo(User, { foreignKey: 'authorId' });  
User.hasMany(Blog, { foreignKey: 'authorId' });
```

```
module.exports = Blog;
```

6. models/index.js

js

CopyEdit

```
const sequelize = require('../config/db');  
const User = require('../user.model');  
const Blog = require('../blog.model');
```

```
// Create associations
```

```
User.hasMany(Blog, { foreignKey: 'authorId' });  
Blog.belongsTo(User, { foreignKey: 'authorId' });
```

```
module.exports = {  
  sequelize,  
  User,  
  Blog,  
};
```

7. controllers/user.controller.js

js

CopyEdit

```
const { User } = require('../models');  
  
exports.register = async (req, res) => {  
  try {  
    const { email, password } = req.body;  
    const user = await User.create({ email, password });  
    res.status(201).json(user);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
};
```

```
exports.getAllUsers = async (req, res) => {  
  const users = await User.findAll();  
  res.json(users);  
};
```

8. controllers/blog.controller.js

js

CopyEdit

```
const { Blog, User } = require('../models');
```

```
exports.createBlog = async (req, res) => {  
  const { title, content, authorId } = req.body;  
  try {  
    const blog = await Blog.create({ title, content, authorId });  
    res.status(201).json(blog);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
};
```

```
exports.getAllBlogs = async (req, res) => {  
  const blogs = await Blog.findAll({ include: User });  
  res.json(blogs);  
};
```

9. routes/user.routes.js

js

CopyEdit

```
const express = require('express');  
const router = express.Router();  
const userController = require('../controllers/user.controller');  
  
router.post('/register', userController.register);  
router.get('/', userController.getAllUsers);  
  
module.exports = router;
```

10. routes/blog.routes.js

js

CopyEdit

```
const express = require('express');
```

```
const router = express.Router();

const blogController = require('../controllers/blog.controller');

router.post('/', blogController.createBlog);
router.get('/', blogController.getAllBlogs);

module.exports = router;
```

11. app.js

js

CopyEdit

```
const express = require('express');
const { sequelize } = require('./models');

const app = express();
app.use(express.json());

app.use('/api/users', require('./routes/user.routes'));
app.use('/api/blogs', require('./routes/blog.routes'));

// Sync DB
sequelize.sync({ alter: true }) // Or { force: true } to drop & recreate
  .then(() => {
    console.log('DB connected');
    app.listen(3000, () => {
      console.log('Server running on http://localhost:3000');
    });
  })
  .catch(err => console.error('Error connecting to DB:', err));
```

 **Test With Postman**

POST /api/users/register

json

CopyEdit

```
{  
  "email": "satish@example.com",  
  "password": "123456"  
}
```

POST /api/blogs

json

CopyEdit

```
{  
  "title": "My First Blog",  
  "content": "Hello Sequelize!",  
  "authorId": 1  
}
```

Q1: Why this index.js file and what is its use?

```
const sequelize = require('../config/db');
```

```
const User = require('../user.model');
```

```
const Blog = require('../blog.model');
```

```
// Create associations
```

```
User.hasMany(Blog, { foreignKey: 'authorId' });
```

```
Blog.belongsTo(User, { foreignKey: 'authorId' });
```

```
module.exports = {
```

```
  sequelize,
```

```
  User,
```

```
  Blog,
```

```
};
```

✅ Purpose of index.js in the models/ folder:

1. Centralized Access Point:

- Instead of importing each model individually everywhere (require('./models/user.model'), etc.), you can just do:

```
js
```

CopyEdit

```
const { User, Blog } = require('./models');
```

2. Model Initialization:

- It initializes all models and sets up associations between them in one place.

3. Clean Code Organization:

- Keeps your code DRY and organized by handling all models and associations in a single file.

4. Auto-loading in Future:

- In more complex setups, you might dynamically load models from the folder and register them here.

Q2: Why these lines in blog model?

```
js
```

CopyEdit

```
Blog.belongsTo(User, { foreignKey: 'authorId' });
```

```
User.hasMany(Blog, { foreignKey: 'authorId' });
```

✅ Meaning & Purpose of these Associations:

Let's say you have:

- A User table
- A Blog table with authorId pointing to User.id

You want to define that:

- **One user can write many blogs** → User.hasMany(Blog)
- **Each blog belongs to one user** → Blog.belongsTo(User)

🔍 So why both are needed?

Sequelize needs **both directions** to:

- Allow you to use .getBlogs() on a User instance.

- Allow you to use `.getUser()` on a `Blog` instance.
- Enable proper foreign key constraints and eager loading (`include: User` or `include: Blog`).

💡 **Without these:**

- Sequelize won't understand the relationships between tables.
- You lose ORM features like:

js

CopyEdit

```
const blogs = await Blog.findAll({ include: User });
```

```
const user = await blog.getUser();
```

✅ **Summary:**

Line	Purpose
<code>index.js</code>	Central place to register models + setup associations
<code>User.hasMany(Blog)</code>	A user can author many blogs
<code>Blog.belongsTo(User)</code>	Each blog has one author